# Sri Lanka Institute of Information Technology

# SE4010 – CTSE

# Assignment 2 – Machine Learning

**Team Size:** 4 Students

## Project Overview

Your objective is to design, build, and deploy a locally-hosted Multi-Agent System (MAS) that automates a complex, multi-step problem.

You have complete freedom to choose the industry vertical or problem domain for your project. Whether you choose to automate a process in Finance, Healthcare, E-Commerce, Education, Entertainment, or internal Software Engineering workflows, your system will be graded strictly on how well it implements the core architectural components of Agentic AI.

Rather than building a generic conversational chatbot, your system must function as an autonomous team of agents that can perceive a problem, reason about it, use tools to interact with the environment, and synthesize a concrete solution.

## Technical Constraints (Zero-Cost & Local)

To ensure data privacy and zero cloud costs, this system must run entirely on your local machines:

- **The LLM Engine:** You must use local Small Language Models (SLMs) via **Ollama** (e.g., llama3:8b, phi3, qwen).
- **The Orchestrator:** Use an open-source framework like **LangGraph**, **CrewAI**, or **AutoGen** to manage the state and routing.
- **Execution:** The system must run locally without requiring paid API keys (OpenAI, Anthropic, etc., are strictly prohibited for this assignment).

## Development Guidelines: What the System Must Have

Regardless of the domain you choose, your final Multi-Agent System must demonstrate the following architectural components:

1. **Multi-Agent Orchestration:** The system must utilize at least 3 to 4 distinct agents interacting with one another (e.g., utilizing a Coordinator-Worker-Delegator or sequential pipeline model).
2. **Tool Usage:** Agents cannot rely solely on their internal LLM knowledge. The system must include custom Python tools that allow agents to interact with the real world (e.g., reading/writing local files, executing terminal commands, querying a local database, or calling free public APIs).
3. **State Management:** The system must demonstrate how "global state" (data) is securely passed from one agent to the next without losing context.
4. **LLMOps/AgentOps & Observability:** You must implement a logging or tracing mechanism to record the specific inputs, tool calls, and outputs of the agents during execution.

## Individual Requirements: What Every Student Must Do

To ensure all team members are exposed to the full spectrum of Agentic AI development, there are no "project managers" or "documentation only" roles. **Every individual student in the group is required to:**

1. **Build an Agent:** Design the system prompt, constraints, and persona for at least one specific agent in the swarm.
2. **Build a Tool:** Write at least one custom Python tool (with strict type hinting and docstrings) that your specific agent will use.
3. **Implement Testing/Evaluation:** Write an automated evaluation script (e.g., property-based testing or an "LLM-as-a-Judge" test) to validate that your specific agent's output is accurate and secure.

## Deliverables

### Source Code Repository

Students should submit a GitHub / GitLab repository containing the complete implementation.

- MAS implementation using LangGraph / CrewAI / AutoGen
- Implementation of 3–4 agents
- Custom Python tools
- Testing / evaluation scripts

### Demo Video

- You should create a demonstration video showing how the application runs and its key functionalities.
- The video should clearly illustrate the workflow of the system.
- The duration of the video must not exceed 4-5 minutes. DO NOT GO BEYOND 5 MINUTES

### Technical Report (4-8 pages Do not go above 8)

A structured report explaining the design and engineering decisions of the system.

- Should explain the problem domain
- System architecture including multi-agent architecture, agent roles and responsibilities and workflow diagram
- Agent Design with system prompts, constraints and reasoning logic including Interaction strategy
- Description of custom tools - APIs / file interactions and example tool usage
- State Management - how context is passed between agents and global state structure
- Evaluation methodology, testing scripts and performance or reliability analysis
- Link to the GitHub repository
- As each student must build one agent, and one tool, students should submit proof of their contributions.
    - Agent developed
    - Tool implemented
    - Challenges faced
- Develop a single, unified testing harness as a group, but each student must contribute the specific test cases and assertions required to validate their own individual agent's output.

## Assessment Criteria

| Criteria & Weight | Excellent (90-100%) | Very Good (80-89%) | Good (70-79%) | Average (60-69%) | Below Average (50-59%) | Poor (<50%) |
|---|---|---|---|---|---|---|
| **Problem Definition & System Architecture (10%)** | Exceptionally clear problem domain. Architecture diagram is highly professional, accurately detailing agents, tools, and workflows. | Clear problem definition. Architecture diagram is accurate and covers all main components but lacks minor details. | Problem is well-defined. Diagrams show basic structure but may have slight inaccuracies in workflow representation. | Problem is somewhat generic. Architecture diagram is basic and misses key interactions. | Vague problem definition. Diagram is incomplete or confusing. | Unclear problem domain. Missing or fundamentally flawed architecture diagram. |
| **Multi-Agent Architecture & Orchestration (15%)** | Flawless orchestration of 3-4 agents using frameworks like LangGraph/CrewAI. Perfect delegation and interaction strategy. | Strong orchestration of 3-4 agents. Interactions are logical and mostly efficient, with minor room for optimization. | Good orchestration; agents interact as expected, but the workflow could be streamlined or relies too heavily on sequential steps. | Basic orchestration. Agents function but interaction feels forced or loosely connected. | Poor orchestration. Agents frequently fail to pass tasks correctly or overlap in responsibilities. | Agents do not interact; acts as a single agent or fails to utilize the chosen framework. |
| **Tool Development & Integration (10%)** | Custom tools are seamlessly integrated, highly relevant to the domain, and handle complex real-world interactions perfectly. | Tools are well-integrated and relevant, effectively handling standard interactions. | Tools are integrated and function correctly, but may be simplistic or only cover basic use cases. | Tools are minimally integrated. They function but provide limited value to the overall system. | Tools are poorly integrated, frequently fail, or are barely used by the agents. | No custom tools integrated, or tools do not function at all. |
| **State Management & Observability (10%)** | Global state is perfectly preserved between agents without context loss. Exceptional execution tracing/logging implemented. | Strong state management with minimal context loss. Good logging mechanism that tracks inputs/outputs clearly. | Adequate state management, though some minor context may be dropped. Basic logging implemented but lacks detail. | State management struggles with complex data. Logging is minimal (e.g., standard print statements only). | State is frequently lost between agent handoffs. Observability is nearly non-existent. | State management fails entirely. No tracking or logging implemented. |
| **System Demonstration (5%)** | Highly polished 3-4 min video. Flawlessly illustrates the workflow and key functionalities running locally. | Clear and concise video demonstrating all key functionalities within the time limit. | Good video, demonstrates the system working but narrative or workflow explanation is slightly unclear. | Average video. System works but presentation is rushed, too long, or hard to follow. | Below average video. Demonstration is incomplete or fails to show the full workflow. | No video submitted, or the video does not show a working system. |

| | | | | | | |
|---|---|---|---|---|---|---|
| **Testing & Evaluation (10%)** | Comprehensive evaluation script (property-based or robust LLM-as-a-judge) perfectly validating agent accuracy and security. | Strong evaluation script that effectively tests the main outputs and constraints of the agent. | Adequate evaluation script. Tests basic functionality but misses edge cases or security validations. | Basic evaluation script. Only tests simple "happy path" scenarios. | Weak evaluation script. Fails to properly validate the agent's output. | No testing script submitted, or the script does not work. |

| **Criteria & Weight** | **Excellent (90-100%)** | **Very Good (80-89%)** | **Good (70-79%)** | **Average (60-69%)** | **Below Average (50-59%)** | **Poor (<50%)** |
|---|---|---|---|---|---|---|
| **Individual Agent Design (20%)** | Exceptional prompt engineering. Persona and constraints perfectly align with SLM capabilities, resulting in zero hallucinations. | Strong agent design. Prompts and constraints are effective, guiding the SLM well with very rare hallucinations. | Good agent design. The agent performs its task, but prompts could be tighter to prevent occasional minor errors. | Basic agent design. Persona is generic; SLM occasionally loses focus or struggles with the task. | Weak agent design. Prompts are inadequate, leading to frequent hallucinations or task failures. | Agent design is fundamentally flawed or missing. |
| **Individual Custom Tool (20%)** | Flawless Python tool with strict type hinting, robust error handling, and highly descriptive docstrings. | Well-written tool with good type hinting and docstrings. Handles standard errors well. | Functional tool with basic type hinting and docstrings. May lack robust error handling. | Tool works under ideal conditions but lacks proper docstrings, type hinting, or error handling. | Tool frequently breaks or is poorly coded, hindering the agent's ability to use it. | Tool is non-functional, missing, or copied without understanding. |